

Atividade acadêmica: APIs e integrações - conceitos, tipos, ferramentas, exemplos

Disciplina: Programação para Web

Curso: Análise e desenvolvimento de Sistemas

Introdução

Na era da transformação digital e dos sistemas distribuídos, a integração entre aplicações heterogêneas tornou-se um dos pilares estratégicos da Engenharia de Software. Historicamente, as organizações dependiam de arquiteturas monolíticas isoladas ou de integrações ponto a ponto engessadas. Contudo, a necessidade de escalabilidade, reuso de código e interoperabilidade impulsionou a adoção de APIs (Application Programming Interfaces) como a espinha dorsal de ecossistemas tecnológicos modernos (Bogner et al., 2023).

As APIs atuam como contratos formais de comunicação que abstraem a complexidade interna de um sistema, expondo apenas os pontos de acesso (endpoints) necessários para a troca segura e estruturada de dados. Seja no contexto de arquiteturas de microserviços, aplicações mobile, ecossistemas de Open Finance ou soluções em nuvem (iPaaS), o domínio dos conceitos, padrões e ferramentas de integração é requisito fundamental para tecnólogos em Análise e Desenvolvimento de Sistemas (Vukotic et al., 2024).

1. Conceitos fundamentais

1.1. Conceito de API

Uma API é um conjunto formal de regras, protocolos e definições que permite a comunicação entre dois softwares distintos, sem que um precise conhecer os detalhes internos de implementação do outro. Na prática, ela funciona como um contrato: expõe operações (endpoints, métodos ou procedimentos), define os

formatos de entrada e saída de dados e estabelece as regras de autenticação e autorização para o consumo do serviço. Embora o conceito de API não seja exclusivo da web (existindo também para sistemas operacionais, bibliotecas e hardware), o foco deste trabalho está nas Web APIs. Tratam-se de interfaces expostas via protocolos de rede, majoritariamente HTTP/HTTPS, que viabilizam a troca de dados entre aplicações pela internet ou em redes corporativas.

1.2. O papel da API na integração de sistemas

A integração de sistemas consiste no processo de conectar aplicações, bancos de dados e plataformas distintas para que operem de forma coordenada, compartilhando dados e fluxos de negócio. Historicamente, essa integração ocorria por meio de acesso direto aos bancos de dados ou pela troca de arquivos em lote (*batch*), abordagens frágeis e de difícil manutenção. As APIs solucionaram essa limitação ao introduzir uma camada de abstração estável: o sistema consumidor interage exclusivamente com a interface publicada, garantindo que alterações internas no provedor não afetem a integração, desde que o contrato estabelecido seja preservado.

Segundo Khanyi et al. (2024), a adoção combinada de redes de dados e APIs está diretamente associada a ganhos mensuráveis de eficiência operacional, tais como o aumento na velocidade dos processos e a redução de custos. Contudo, quando mal implementadas, essas interfaces tornam-se fontes relevantes de vulnerabilidades, o que reforça a necessidade de tratar governança e segurança como pilares indissociáveis do projeto de integração.

Nesse contexto, o modelo baseado em APIs oferece os seguintes benefícios estruturais:

- **Desacoplamento:** permite que os sistemas evoluam de forma independente, evitando que mudanças internas causem impactos em cadeia.
- **Reusabilidade:** viabiliza que uma mesma API seja consumida por múltiplos clientes, como sistemas web, aplicações móveis e parceiros externos.
- **Escalabilidade:** possibilita o dimensionamento individualizado de componentes específicos da arquitetura conforme a demanda.

- **Governança:** garante o controle centralizado de acessos, frequências de uso e políticas de segurança por meio de *gateways* e catálogos de APIs.

2. Tipos de APIs

Não existe um único padrão arquitetural de API adequado a todos os cenários de integração. A escolha da abordagem ideal depende de fatores como o volume e a natureza dos dados trafegados, os requisitos de desempenho e latência, o nível de desacoplamento desejado entre os sistemas e o ecossistema tecnológico preexistente na organização. A seguir, apresentam-se os principais padrões empregados atualmente.

2.1. REST (Representation State Transfer)

REST é um estilo arquitetural, e não um protocolo, que trata cada elemento do sistema como um recurso identificado por uma URI e manipulado por meio de métodos padrão do protocolo HTTP (GET, POST, PUT, PATCH e DELETE). Trata-se, atualmente, do estilo dominante na integração de sistemas web, impulsionado por sua simplicidade, pelo caráter *stateless* (ausência de retenção de estado entre requisições) e pelo amplo suporte em ferramentas de documentação, testes e segurança. Golmohammadi, Zhang e Arcuri (2023) destacam que a maioria dos serviços web modernos é padronizada pela especificação OpenAPI (OAS), o que viabilizou o surgimento de um ecossistema robusto para testes automatizados, geração de código cliente e documentação interativa.

2.2. SOAP (Simple Object Access Protocol)

SOAP é um protocolo baseado em XML que define um formato rígido para o intercâmbio de mensagens (estruturado em envelopes) e depende de um contrato formal especificado em WSDL (*Web Services Description Language*). Embora tenha perdido espaço para o REST no desenvolvimento web moderno, o SOAP mantém sua relevância em sistemas corporativos legados e setores altamente regulados, como o financeiro. Isso se deve à sua oferta de recursos avançados de segurança nativa (WS-Security), suporte a transações distribuídas e uso de contratos estritamente tipados.

2.3. GraphQL

Desenvolvido pela Meta (anteriormente Facebook) e disponibilizado publicamente em 2015, o GraphQL é uma linguagem de consulta que permite ao cliente solicitar exatamente os campos de dados necessários em uma única requisição. Essa abordagem soluciona os problemas clássicos de *over-fetching* (recebimento de dados em excesso) e *under-fetching* (necessidade de múltiplas chamadas para compor as informações), comuns em APIs REST tradicionais. Estudos comparativos, como o de Quiña-Mera et al. (2023), evidenciam, por meio de testes controlados com dados de e-commerce, que o GraphQL pode apresentar eficiência superior ao REST em cenários caracterizados pelo consumo intensivo de dados relacionais.

3. Ferramentas

O ciclo de vida de uma API, desde a concepção inicial até o consumo em ambiente de produção, é sustentado por um ecossistema abrangente de ferramentas especializadas. O quadro a seguir sintetiza as principais etapas desse processo, destacando as categorias de ferramentas e exemplos de tecnologias empregadas em cada fase da integração.

Categoria	Exemplos de ferramentas	Função na integração
Especificação e design	OpenAPI (Swagger), AsyncAPI, GraphQL SDL	Descrevem contratos de API (endpoints, tipos, autenticação) de forma padronizada e legível por máquina, permitindo geração automática de documentação, SDKs e testes.
Testes e prototipagem	Postman, Insomnia, cURL	Permitem enviar requisições, validar respostas, criar coleções de testes automatizados e simular cenários de integração antes do consumo em produção.
API Gateway	Kong, Apigee, AWS API Gateway, Azure API Management	Centralizam autenticação, limitação de taxa (rate limiting), roteamento, versionamento e observabilidade de múltiplas

		APIs expostas por uma organização.
Middleware / ESB	MuleSoft, IBM App Connect, WSO2	Orquestram e transformam mensagens entre sistemas heterogêneos, aplicando padrões de integração corporativa (EIP).
Mensageria e eventos	Apache Kafka, RabbitMQ, AWS SQS/SNS	Habilitam integração assíncrona e desacoplada entre serviços por meio de filas e tópicos de eventos, essencial em arquiteturas orientadas a eventos.
Segurança	OAuth 2.0, OpenID Connect, JWT	Padrões de autenticação e autorização que garantem que apenas clientes legítimos acessem recursos protegidos de uma API.
Orquestração de contêineres	Docker, Kubernetes, Istio (service mesh)	Suportam a implantação, escalonamento e comunicação segura entre os microsserviços que expõem as APIs.

3.1. Especificação e documentação

A especificação OpenAPI, anteriormente conhecida como Swagger, consolidou-se como o padrão *de facto* para descrever APIs RESTful de forma legível tanto por humanos quanto por máquinas. Gu et al. (2024) demonstram, inclusive, como essa especificação pode ser estendida com anotações semânticas de domínio (ontologias) para viabilizar a descoberta automática e a composição dinâmica dessas interfaces. Essa abordagem vai além da simples documentação estática, impulsionando o desenvolvimento de ecossistemas de integração mais inteligentes.

3.2. Segurança de API

A crescente exposição de dados sensíveis por meio de APIs tornou a segurança um requisito arquitetural indispensável, e não um elemento opcional. Tanveer et al. (2025) sistematizam as principais classes de vulnerabilidade em APIs

RESTful, como falhas de autenticação e autorização, referências diretas a objetos inseguros (IDOR) e falhas de validação de entrada. Os autores propõem uma taxonomia voltada especificamente a falhas de projeto de API, além de avaliar métodos de detecção baseados em análise estática de código e testes de penetração automatizados.

Diante desse cenário, as práticas de segurança mais recomendadas incluem:

- **Uso de OAuth 2.0 e OpenID Connect:** padronização do processo de autenticação e autorização delegada.
- **Emprego de *tokens* JWT (*JSON Web Token*):** utilização de *tokens* assinados digitalmente e com curto tempo de expiração.
- **Validação rigorosa de entrada:** verificação em todos os *endpoints* para mitigar vulnerabilidades e ataques de injeção.
- **Controle de acesso em nível de objeto:** prevenção contra falhas de autorização do tipo IDOR (*Insecure Direct Object References*).
- **Adoção de *API Gateways*:** centralização do controle de taxa (*rate limiting*), do monitoramento e da mitigação de acessos abusivos.

3.3. Orquestração de infraestrutura

Em arquiteturas baseadas em microsserviços, tecnologias de containerização, como o Docker, e de orquestração, como o Kubernetes, são fundamentais para implantar, escalar e manter a alta disponibilidade dos serviços que expõem as APIs. Complementarmente, ferramentas de *service mesh*, a exemplo do Istio, adicionam uma camada de comunicação segura, gerenciável e observável no tráfego interno entre microsserviços, expandindo e fortalecendo as capacidades de controle fornecidas pelo *API Gateway* na borda da aplicação.

4. Exemplos de Integração por meio de API

4.1. Setor bancário: modernização de arquitetura

O estudo de Fauziah e Surantha (2026) ilustra um caso prático de integração no setor financeiro: diante do crescimento expressivo das transações digitais de depósito sem cartão, uma instituição bancária reestruturou seu sistema monolítico para uma arquitetura de microsserviços apoiada em *middleware*. Nessa nova

abordagem, cada funcionalidade do sistema (como validação de transações, autenticação e notificações) passou a ser exposta como um serviço independente, comunicando-se via API. Como resultado, os autores reportaram ganhos significativos em desempenho, escalabilidade e disponibilidade da plataforma.

4.2. Comércio eletrônico: comparação de estilos de API

Quiña-Mera et al. (2023) utilizaram dados abertos da Câmara de Comércio Eletrônico do Equador para comparar, em ambiente controlado de laboratório, o desempenho de uma API REST, uma API GraphQL e uma API GraphQL atuando como uma camada de encapsulamento (*wrapper*) sobre a API REST original. O experimento exemplifica uma estratégia recorrente de integração: sobrepor uma camada GraphQL a um sistema legado REST para modernizar o consumo de dados pelos clientes, eliminando a necessidade de reescrever a infraestrutura de *back-end* preexistente.

4.3. Pequena e médias empresas: eficiência operacional

Khanyi et al. (2024) sintetizam evidências de 49 estudos sobre o uso de redes de dados e APIs em pequenas e médias empresas (PMEs), constatando ganhos médios de aproximadamente 12% na velocidade dos processos e uma redução de 8% nos custos operacionais decorrentes da integração via API. Em contrapartida, os autores alertam para o crescimento expressivo de ataques direcionados a APIs vulneráveis nesse segmento, que frequentemente dispõe de menos recursos para investimentos em segurança cibernética.

4.4. Novos domínios: interoperabilidade no metaverso

Um exemplo de aplicação emergente é apresentado por Traub e Weinberger (2024), que avaliaram a disponibilidade de APIs públicas em quinze plataformas de metaverso, tais como Roblox, VRChat, Decentraland e Second Life. O estudo constatou que apenas parcela dessas plataformas disponibiliza APIs abertas e que, quando presentes, apresentam expressiva heterogeneidade estrutural. Essa falta de

padronização dificulta a interoperabilidade entre os diferentes mundos virtuais, ilustrando um desafio clássico na integração de sistemas pertencentes a ecossistemas tecnológicos em fase de consolidação, nos quais a definição formal de contratos de API torna-se fator decisivo para o sucesso da área.

Conclusão

As APIs deixaram de ser um mero detalhe técnico de implementação e tornaram-se a peça central da estratégia de integração de sistemas nas organizações contemporâneas. A escolha entre os diferentes padrões de API, tais como REST, SOAP, GraphQL, gRPC ou modelos orientados a eventos, deve ser rigorosamente guiada pelos requisitos específicos de cada projeto. Isso inclui o volume e o formato dos dados, a latência exigida, o grau de desacoplamento tolerável e a maturidade do ecossistema tecnológico envolvido, visto que não existe uma solução universalmente superior.

A literatura acadêmica revisada evidencia, ainda, três tendências convergentes no cenário atual: a consolidação da arquitetura de microserviços como padrão dominante para sistemas distribuídos, acompanhada do combate contínuo a antipadrões e da busca pela qualidade arquitetural; o amadurecimento do ecossistema de ferramentas, abrangendo especificação, testes automatizados e *gateways* que profissionalizam a gestão de todo o ciclo de vida da API; e a centralidade da segurança cibernética, na qual vulnerabilidades específicas da camada de integração, a exemplo das falhas de autorização em nível de objeto, se consolidaram como um dos principais vetores de ataque a sistemas corporativos. Compreender esses conceitos, padrões, arquiteturais e ferramentas é, portanto, condição indispensável para o planejamento e a execução de integrações de sistemas robustas, escaláveis e seguras.

Referências

ABGAZ, Yalemisew; MCCARREN, Andrew; ELGER, Peter; SOLAN, David; LAPUZ, Neil; BIVOL, Marin; JACKSON, Glenn; YILMAZ, Murat; BUCKLEY, Jim; CLARKE, Paul. Decomposition of Monolith Applications Into Microservices Architectures: A Systematic Review. *IEEE Transactions on Software Engineering*, v. 49, n. 8, p. 4213–4242, 2023. DOI: 10.1109/TSE.2023.3287297.

CERNY, Tomas; ABDELFAHATTAH, Amr S.; AL MARUF, Abdullah; JANES, Andrea; TAIBI, Davide. Catalog and Detection Techniques of Microservice Anti-Patterns and Bad Smells: A Tertiary Study. *Journal of Systems and Software*, v. 206, artigo 111829, 2023. DOI: 10.1016/j.jss.2023.111829.

FAUZIAH, Rizza; SURANTHA, Nico. Evaluating Middleware Performance in the Transition from Monolithic to Microservices Architecture for Banking Applications. *Electronics*, v. 15, n. 1, artigo 221, 2026. DOI: 10.3390/electronics15010221.

GOLMOHAMMADI, Amid; ZHANG, Man; ARCURI, Andrea. Testing RESTful APIs: A Survey. *ACM Transactions on Software Engineering and Methodology*, v. 33, n. 1, artigo 27, 2023. DOI: 10.1145/3617175.

GU, Haijun; MA, Yingyu; WANG, Siqi; CHEN, Xincheng; SU, Weihua. Semantically Realizing Discovery and Composition for RESTful Web Services. *Computing*, v. 106, n. 7, p. 2361–2387, 2024. DOI: 10.1007/s00607-024-01289-8.

KHANYI, Mduduzi B.; XABA, Sfundu N.; MLOTSHWA, Nokunqoba A.; THANGO, Bonginkosi; MATSHAKA, Lerato. A Roadmap to Systematic Review: Evaluating the Role of Data Networks and Application Programming Interfaces in Enhancing Operational Efficiency in Small and Medium Enterprises. *Sustainability*, v. 16, n. 23, artigo 10192, 2024. DOI: 10.3390/su162310192.

QUIÑANA-MERA, Antonio; GUEVARA-VEGA, Cathy; CAIZA, José; MISE, José; LANDETA, Pablo. REST, GraphQL, and GraphQL Wrapper APIs Evaluation. A Computational Laboratory Experiment. In: *PROCEEDINGS OF INTERNATIONAL CONFERENCE ON INFORMATION TECHNOLOGY AND APPLICATIONS*. Lecture Notes in Networks and Systems, v. 614. Singapura: Springer, 2023. p. 397–407. DOI: 10.1007/978-981-19-9331-2_34.

TANVEER, Fatima; IRADAT, Faisal; IQBAL, Waseem; AHMAD, Awais. Towards Secure APIs: A Survey on RESTful API Vulnerability Detection. *Computers, Materials & Continua*, v. 84, n. 3, p. 4223–4257, 2025. DOI: 10.32604/cmc.2025.067536.

TRAUB, Marius; WEINBERGER, Markus. APIs in the Metaverse — A Systematic Evaluation. *Virtual Worlds*, v. 3, n. 2, p. 157–170, 2024. DOI: 10.3390/virtualworlds3020008.

VELEPUCHA, Victor; FLORES, Pamela. A Survey on Microservices Architecture: Principles, Patterns and Migration Challenges. *IEEE Access*, v. 11, p. 88339–88358, 2023. DOI: 10.1109/ACCESS.2023.3305687.